



**UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

**HENRIQUE NORONHA
LAURO OLIVEIRA MOTA
JOÃO VICTOR MELO DO NASCIMENTO
HÁTILAN CAIO ALVES FONTES**

**ANÁLISE COMPARATIVA DE GERENCIAMENTO DE RECURSOS DO
SO: ABLETON LIVE VS. FL STUDIO**

PALMAS (TO)

2026

Henrique Noronha
Lauro Oliveira Mota
João Victor Melo do Nascimento
Hátilan Caio Alves Fontes

**Análise Comparativa de Gerenciamento de Recursos do SO: Ableton Live vs.
FL Studio**

Relatório de Sistemas Operacionais
apresentado à Universidade Federal do
Tocantins para obtenção do título de Bacharel
em Ciência da Computação.

PALMAS (TO)

2026

SUMÁRIO

1	INTRODUÇÃO	0
2	REFERENCIAL TEÓRICO	1
2.1	Processos, <i>threads</i> e escalonamento	1
2.2	Interrupções, latência e drivers de áudio	1
2.3	Memória, paginação e estratégias de alocação	1
3	METODOLOGIA	2
4	RESULTADOS	3
4.1	Testes 1 e 2: consumo em repouso e carga I/O-bound	3
4.2	Teste 3: escalonamento de threads	3
4.3	Teste 4: latência com driver nativo	4
4.4	Teste 5: estresse CPU-bound e ponto de ruptura	4
5	DISCUSSÃO	5
5.1	Padrões arquiteturais (T1-T3)	5
5.2	Comportamento sob carga e latência (T4-T5)	5
6	CONCLUSÃO	6
	REFERÊNCIAS	7

1 INTRODUÇÃO

Os Sistemas Operacionais (SO) modernos precisam lidar com arquiteturas de múltiplos núcleos, memória virtual e interrupções em escala de milissegundos. Nesse cenário, muitos computadores pessoais acabam sendo usados como Estações de Trabalho de Áudio Digital (DAWs, do inglês *Digital Audio Workstations*), cuja performance não depende só da força bruta do processador, mas também de como o SO gerencia as *threads*, aloca memória e cuida das operações de I/O. O grande desafio desse tipo de aplicação é operar sob restrições de tempo real. Segundo Silberschatz, Galvin e Gagne (2015), as DAWs se enquadram como sistemas de tempo real não crítico (*soft real-time*): um atraso no atendimento não causa falha grave, mas se manifesta como falhas e artefatos sonoros. Para evitar isso, o SO precisa garantir que as *threads* de áudio tenham preferência sobre os processos comuns.

A partir disso, este trabalho faz uma análise comparativa do gerenciamento de recursos do SO entre duas das DAWs mais usadas no mercado: o Ableton Live e o FL Studio. A ideia é mapear como cada arquitetura afeta o consumo de CPU, o comportamento *multithread*, o uso de memória RAM e a latência de I/O sob as mesmas condições de estresse.

O restante do trabalho está organizado em cinco capítulos. O Capítulo 2 apresenta o referencial teórico sobre processos, *threads*, escalonamento, interrupções, latência de I/O e gerenciamento de memória, com foco no Windows NT. O Capítulo 3 descreve a metodologia, as ferramentas usadas (PerfMon, Process Explorer e LatencyMon) e o protocolo dos cinco testes. O Capítulo 4 reúne os resultados de cada teste, e o Capítulo 5 discute as diferenças arquiteturais observadas entre as duas DAWs. Por fim, o Capítulo 6 sintetiza as principais conclusões do estudo.

2 REFERENCIAL TEÓRICO

2.1 Processos, *threads* e escalonamento

Um processo é um programa em execução, com espaço de endereçamento próprio e os recursos a ele atribuídos pelo SO (SILBERSCHATZ; GALVIN; GAGNE, 2015). Dentro dele podem coexistir várias *threads*, que compartilham esse espaço mas mantêm pilha e registradores próprios (TANENBAUM; BOS, 2016). Como a troca de contexto entre *threads* é mais barata do que entre processos, motores de áudio recorrem ao *multithreading* para distribuir o processamento entre os núcleos do hardware. O escalonador do Windows NT é preemptivo baseado em prioridades, com 32 níveis: a faixa de usuário ocupa 1–15 e a faixa de tempo real, 16–31 (STALLINGS, 2017). Os valores observados aqui ficam todos na faixa de usuário: *Normal* (8), *Above Normal* (10) e *High* (12–13). As DAWs se enquadram como sistemas *soft real-time*: um *buffer* não preenchido a tempo gera *dropout* audível, mas o sistema continua estável (SILBERSCHATZ; GALVIN; GAGNE, 2015).

2.2 Interrupções, latência e drivers de áudio

Uma interrupção de hardware é um sinal assíncrono pelo qual um periférico avisa o processador de que precisa de atenção imediata (SILBERSCHATZ; GALVIN; GAGNE, 2015). A soma da latência da Rotina de Serviço de Interrupção (ISR) com a troca de contexto até a *thread* destino é a latência de despacho (STALLINGS, 2017). Os drivers de áudio operam no modelo *pull callback*: o hardware pede periodicamente um bloco de N amostras e o motor tem até o prazo do próximo bloco para entregá-lo (FONS, 2010). A relação entre *buffer* e latência é direta, $L = N/f_s$, onde f_s é a taxa de amostragem; reduzir N diminui a latência mas aumenta a frequência de interrupções (KIM; HAHN, 2003). No Windows, o caminho WDM/DirectSound passa pelo *Kernel Audio Mixer* (KMixer), que mistura fluxos de vários aplicativos e precisa ser acordado a cada ciclo de *buffer*; já o protocolo ASIO dá ao driver do fabricante acesso direto ao hardware, ignorando o KMixer (Microsoft Corporation, 2023a).

2.3 Memória, paginação e estratégias de alocação

Cada processo recebe do SO um espaço de endereçamento virtual contíguo e isolado (SILBERSCHATZ; GALVIN; GAGNE, 2015), com paginação por demanda: uma página só é mapeada a um quadro físico quando é acessada pela primeira vez (*page fault*) (TANENBAUM; BOS, 2016). Para áudio em tempo real, uma falta no meio do processamento de um bloco é cara, porque a *thread* fica bloqueada enquanto o prazo do *buffer* continua correndo (KIM; HAHN, 2003). Por isso, DAWs comerciais costumam usar *eager allocation* (que reserva memória logo na inicialização e elimina faltas no caminho crítico) em vez de *lazy allocation* (que mantém pegada mínima mas introduz latência na primeira alocação). O contador *Private Bytes* do Windows, usado neste trabalho, mede a memória virtual comprometida só pelo processo, sem contar DLLs e regiões compartilhadas (STALLINGS, 2018).

3 METODOLOGIA

O estudo adota uma abordagem quantitativa e comparativa para analisar o gerenciamento de recursos de *Digital Audio Workstations* (DAWs) sob a ótica de Sistemas Operacionais. O foco é a interação entre o *kernel* do Windows 11 (build 26200, x64) e os motores de áudio do Ableton Live 12 e do FL Studio 24. Os experimentos foram feitos em um notebook ASUS Vivobook K3605ZF com processador Intel Core i5-12500H (12 núcleos, 16 *threads*, base 3,1 GHz), 16 GB de RAM DDR4 e GPU NVIDIA (driver `nvlddmkm.sys` v581.29). Para preservar as escolhas arquiteturais nativas de cada fabricante, o FL Studio rodou com o driver FL Studio ASIO e o Ableton Live rodou com MME/DirectX. O *Fruity Limiter* do canal Master do FL Studio foi removido antes dos testes para manter paridade na cadeia de sinal. O Windows Defender ficou ativo em todas as sessões, em condições iguais para as duas DAWs.

A coleta usou três ferramentas: o Performance Monitor (PerfMon) para capturar contadores do *kernel* a 1 Hz (% Tempo de Processador, Private Bytes e Contagem de Threads), exportados via `relog` para `.csv` (Microsoft Corporation, 2023b); o Process Explorer da Sysinternals para inspecionar *threads* individuais, prioridades e consumo de CPU; e o LatencyMon v7.31 (BERNSEE, 2023) para medir latência *interrupt-to-process*, tempos de DPC/ISR e *hard pagefaults*.

O protocolo experimental foi composto por cinco testes sequenciais. O Teste 1, de repouso, coletou dados durante 2,5 minutos com a DAW aberta e sem projeto carregado. O Teste 2, de carga *I/O-bound*, adicionou a reprodução simultânea em *loop* de 20 trilhas AIFF (PCM não comprimido) por 3 minutos. No Teste 3, de escalonamento, o Process Explorer foi usado para inspecionar a composição, a prioridade e a distribuição das *threads* com o projeto ativo. No Teste 4, de latência, o LatencyMon foi executado por 2 min 50 s sobre o mesmo projeto, registrando DPCs, ISRs e *pagefaults*. Por fim, no Teste 5, de estresse CPU-bound, foram inseridas cinco instâncias do reverb nativo de cada DAW no canal Master, e o tamanho do *buffer* foi reduzido aos poucos de 4096 até 128 *samples*. O ponto de ruptura (*buffer underrun*) foi identificado pelo aparecimento de *dropouts* audíveis.

O trabalho se caracteriza como um estudo de caso exploratório, sem pretensão de generalização estatística. Para extrair intervalos de confiança a partir das séries de 145 a 178 amostras coletadas por sessão, foi aplicada a técnica de *bootstrap* com reamostragem por reposição proposta por Efron (1979), gerando $B = 10\,000$ reamostragens por métrica, valor recomendado para estimativas estáveis de intervalos percentílicos em $B \geq 1\,000$, com 10 000 como escolha conservadora usual em estudos empíricos. Os intervalos de confiança de 95% são reportados ao lado das médias.

4 RESULTADOS

4.1 Testes 1 e 2: consumo em repouso e carga I/O-bound

Os Testes 1 e 2 avaliaram o comportamento de CPU, memória e *threads* em dois cenários consecutivos: DAW em repouso (sem projeto) e com reprodução simultânea de 20 trilhas AIFF em *loop*. Os dados, amostrados a 1 Hz via PerfMon durante 2,5 e 3 minutos respectivamente, estão consolidados nas Tabelas 1 e ??.

Tabela 1 – Médias dos Testes 1 (repouso) e 2 (20 trilhas AIFF). Valores entre parênteses indicam o máximo observado.

Métrica	T1 Ableton	T1 FL Studio	T2 Ableton	T2 FL Studio
% CPU média (máx)	10,95 (33,73)	12,38 (37,72)	3,74 (20,11)	4,80 (31,16)
<i>Private Bytes</i> (MB)	536,8	95,8	601,7	147,6
<i>Threads</i> ativas	54,8	44,0	55,3	27,0

Em repouso, o FL Studio consumiu CPU média um pouco maior (12,38% contra 10,95%), mas o Ableton Live alocou 5,6 vezes mais memória (536,8 MB contra 95,8 MB) e quase o dobro de *threads* (54,8 contra 44,0). Com as 20 trilhas tocando, as duas DAWs reduziram a média de CPU (o Ableton para 3,74% e o FL Studio para 4,80%), a memória cresceu pouco (+65 e +52 MB) e o FL Studio reduziu suas *threads* de 44 para 27 enquanto o Ableton ficou praticamente igual. A Figura 1 mostra a evolução da CPU nos dois cenários.

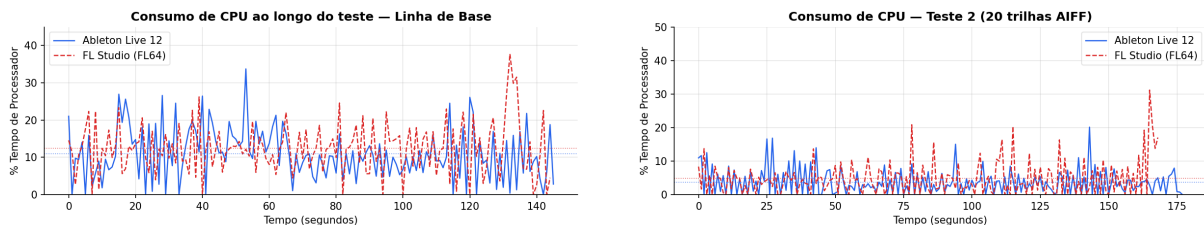


Figura 1 – Consumo de CPU em repouso (esq.) e durante a reprodução de 20 trilhas AIFF (dir.).

4.2 Teste 3: escalonamento de threads

A inspeção via Process Explorer com o projeto em reprodução mostrou diferenças na composição e na prioridade das *threads* (Tabela 2). O FL Studio roda com prioridade base 10 (*Above Normal*) e dinâmica 12 (*High*), enquanto o Ableton Live fica em base 8 (*Normal*) e dinâmica 10. Nenhuma das duas atinge o nível *Time Critical* (15). Na composição, 14 das 29 *threads* do FL Studio pertencem ao QuickFontCache (renderização da interface) e só 5 ao FLEngine_x64 (motor de áudio). No Ableton Live, cerca de 30 das 55 *threads* vêm do ucrtbase.dll, que é o *thread pool* do C runtime.

Tabela 2 – Composição e prioridade das *threads* no Teste 3, obtidas via Process Explorer.

Parâmetro	FL Studio (FL64)	Ableton Live 12
Total de <i>threads</i>	29	55
Prioridade base (thread principal)	10 (Above Normal)	8 (Normal)
Prioridade dinâmica	12 (High)	10 (Above Normal)
CPU da thread principal	0,93%	0,75%
Thread <i>Time Critical</i> (15)?	Não	Não
Módulo dominante	QuickFontCache	ucrtbase
Thread de driver de áudio	ilwasapi2asio_x64	DSOUND.dll

4.3 Teste 4: latência com driver nativo

O LatencyMon classificou as duas DAWs como aptas, com latência máxima *interrupt-to-process* de 805,40 μs no Ableton Live e 939,80 μs no FL Studio, ambas abaixo do limite de 1000 μs adotado pela ferramenta como referência para áudio em tempo real (BERNSEE, 2023). O maior DPC do FL Studio veio do driver NVIDIA `nvlddmkm.sys` (647 μs) e o do Ableton Live veio do `tcpip.sys` (449 μs). Em ambas as sessões, o processo com mais *hard pagefaults* foi o `mmpeng.exe` do Windows Defender (306 no Ableton e 378 no FL Studio).

4.4 Teste 5: estresse CPU-bound e ponto de ruptura

Com cinco instâncias de reverb no canal Master e as 20 trilhas em reprodução, reduzimos o *buffer* aos poucos até começar a aparecer *dropouts* audíveis. O FL Studio com ASIO se manteve estável até 256 *samples* (5,8 ms) e quebrou em 128 *samples* (2,9 ms). Já o Ableton Live com MME/DirectX começou a falhar em 1024 *samples* (23,2 ms), com degradação crescente nos valores menores. A Figura 2 ilustra esse comportamento.

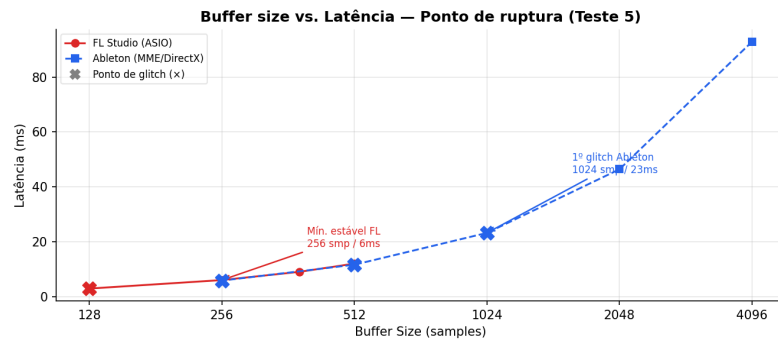


Figura 2 – Tamanho do *buffer* e latência correspondente no Teste 5. O marcador $\times$ indica o ponto de *dropout* em cada DAW.

5 DISCUSSÃO

5.1 Padrões arquiteturais (T1-T3)

Os Testes 1 a 3 deixam claro que as duas DAWs seguem caminhos opostos desde a inicialização. O Ableton Live usa *eager allocation*, com 537 MB e 55 *threads* já em repouso, o que mantém a latência de despacho baixa desde o início (SILBERSCHATZ; GALVIN; GAGNE, 2015). O FL Studio adota *lazy allocation*, com 96 MB e 44 *threads* em repouso, e só amplia esses recursos quando o projeto é carregado (TANENBAUM; BOS, 2016). A queda de *threads* do FL Studio (de 44 para 27) ao carregar o projeto sugere uma consolidação de *threads* genéricas em outras mais especializadas para áudio. No escalonamento, o FL Studio compensa o menor número de *threads* com prioridade mais alta (base 10, dinâmica 12), mas boa parte desse orçamento é consumido pelo `QuickFontCache` da interface, e não pelo motor de áudio. O Ableton Live, mesmo em prioridade *Normal* (8), distribui a carga em 55 *threads* via *thread pool* do `ucrtbase`, o que favorece a escalabilidade nos 16 *threads* lógicos do i5-12500H. Nenhuma das duas usa prioridade *Time Critical* (15), o que confirma o enquadramento delas como *soft real-time* (STALLINGS, 2017).

5.2 Comportamento sob carga e latência (T4-T5)

Em ambiente controlado, o LatencyMon classificou as duas DAWs como aptas, com latências máximas abaixo do limite de 1000 μ s. O maior DPC do FL Studio veio do driver NVIDIA (`nvlddmkm.sys`), o que indica que a GPU gera interrupções longas e disputa o escalonador com o motor de áudio, independentemente da DAW. Em uma sessão anterior, com o Windows Defender em surto de varredura, a latência do FL Studio chegou a 5274 μ s e o sistema foi classificado como não apto, o que mostra que interferências externas podem ter impacto até maior do que as próprias diferenças entre DAWs. Sob estresse CPU-bound, o FL Studio com ASIO se manteve estável até 256 *samples* (6 ms), confirmando o benefício do acesso direto ao hardware: como o ASIO não passa pelo KMixer, reduz o número de interrupções por ciclo de *buffer* (Microsoft Corporation, 2023a). Já o Ableton Live com MME/DirectX começou a apresentar *dropouts* em 1024 *samples*, quatro vezes acima do ponto de ruptura do FL Studio, em parte por causa do caminho pelo KMixer (Microsoft Corporation, 2023a; KIM; HAHN, 2003). Vale notar que essa comparação reflete mais a diferença entre os drivers do que entre os motores de áudio em si; num cenário em que as duas DAWs usassem ASIO, o resultado provavelmente seria diferente.

6 CONCLUSÃO

Este estudo mostrou que, do ponto de vista do gerenciamento de recursos do SO, o Ableton Live e o FL Studio seguem caminhos arquiteturais opostos. O Ableton Live aposta na alocação antecipada (*eager allocation*), reservando memória e criando um *pool* grande de *threads* desde o início, o que garante baixa latência de despacho. Já o FL Studio adota alocação sob demanda (*lazy allocation*) e tenta compensar a menor quantidade de *threads* com prioridade de escalonamento mais alta, embora boa parte desse esforço acabe sendo consumido pela renderização da interface (**QuickFontCache**), e não pelo motor de áudio. Em ambiente controlado, as duas foram classificadas como aptas pelo LatencyMon, com latências bem abaixo do limite de $1000\ \mu s$, e o uso de drivers nativos (especialmente ASIO) mostrou-se importante para reduzir a sobrecarga de CPU. Em ambas, o impacto de processos externos como o Windows Defender em surto de varredura chegou a ser maior do que as próprias diferenças entre elas.

REFERÊNCIAS

- BERNSEE, M. **ASIO4ALL: Universal ASIO Driver for WDM Audio**. [S.l.], 2023. Disponível em: <<https://www.asio4all.org>>. Acesso em: 26 maio 2026.
- EFRON, B. Bootstrap methods: Another look at the jackknife. **The Annals of Statistics**, v. 7, n. 1, p. 1–26, 1979.
- FONS, J. Real-time audio processing on general purpose operating systems. In: **Linux Audio Conference**. Utrecht: [s.n.], 2010.
- KIM, Y.; HAHN, M. Improving real-time audio performance under general purpose operating systems. **IEICE Transactions on Information and Systems**, E86-D, n. 1, p. 192–196, 2003.
- Microsoft Corporation. **Audio Drivers for Windows: ASIO Architecture**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-hardware/drivers/audio>>. Acesso em: 26 maio 2026.
- Microsoft Corporation. **Windows Performance Monitor: Technical Reference**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/perfmon>>. Acesso em: 26 maio 2026.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. Tradução de: Operating System Concepts, 9th ed.
- STALLINGS, W. **Operating Systems: Internals and Design Principles**. 9. ed. Hoboken: Pearson, 2017.
- STALLINGS, W. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2018. Tradução de: Computer Organization and Architecture, 10th ed.
- TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução de: Modern Operating Systems, 4th ed.